Database Normalization Activity

Construction Project Dataset: 1NF → 2NF → 3NF → BCNF → 4NF

Group 5

Tito · Cedric · Ange · Christian

Source file: big3_construction_raw_data.csv (15 rows, 26 columns)

Contents

Summary.....
Step 1: Understanding the Raw Data.....
1.1 Real-world entities.....
1.2 Columns repeated across many rows.....
1.3 Multi-valued (non-atomic) cells.....
1.4 Candidate identifiers.....
1.5 Functional dependencies observed.....
1.6 Relationships.....
1.7 Anomalies caused by the raw design.....
1.8 Assumptions.....
Stage 1: First Normal Form (1NF).....
Stage 2: Second Normal Form (2NF).....
Stage 3: Third Normal Form (3NF).....
Stage 4: Boyce-Codd Normal Form (BCNF).....
Stage 5: Fourth Normal Form (4NF).....
Primary and Foreign Key Summary (Final Design).....
Relationship diagram (text form).....
Data-quality notes for the client.....
Group Roles and Contributions.....

(If the table of contents is empty, right-click it in Word and choose "Update Field".)

Summary

We normalized the raw construction file one stage at a time. Each stage folder in the submission holds the CSV files for that stage. Every CSV represents exactly one entity or one relationship.

Stage	Files	What changed
1NF	1	Split every |-separated cell into atomic values by creating new rows (15 raw rows → 280 rows).
2NF	6	Removed partial dependencies: project, worker, supplier, material-price and equipment-cost facts moved to their own tables.
3NF	8	Removed transitive dependencies: client and supervisor details moved out of the projects table.
BCNF	8	Checked every determinant. All are candidate keys, so no decomposition was needed (the check is documented).
4NF	11	Split the all-key link table along its independent multi-valued facts: project workers, worker skills, worker certifications, supplier phones.

Final 4NF schema (**bold** = primary key, → = foreign key to the referenced table):

projects(**ProjectID**, ProjectName, ProjectType, StartDate, EndDate, SiteAddress, SiteCity, SiteState, ClientID → clients, SupervisorID → supervisors)
clients(**ClientID**, ClientName, ClientPhone, ClientEmail, ClientCity)
supervisors(**SupervisorID**, SupervisorName, SupervisorPhone)
workers(**WorkerID**, WorkerName, WorkerPhone, WorkerHourlyRate)
suppliers(**SupplierID**, SupplierName, SupplierCity)
project_workers(**ProjectID** → projects, **WorkerID** → workers)
worker_skills(**WorkerID** → workers, **Skill**)
worker_certifications(**WorkerID** → workers, **Certification**)
supplier_phones(**SupplierID** → suppliers, **SupplierPhone**)
project_material_supply(**ProjectID** → projects, **SupplierID** → suppliers, **MaterialSupplied**, MaterialUnitCost)
project_equipment(**ProjectID** → projects, **EquipmentUsed**, EquipmentRentalCost)

Step 1: Understanding the Raw Data

The raw file has 15 rows and 26 columns. Each row mixes facts about a project, its site, its client, its supervisor, one worker, one supplier, the materials that supplier delivered and the equipment used.

1.1 Real-world entities

Entity	Columns that describe it
Project	ProjectID, ProjectName, ProjectType, StartDate, EndDate
Site (location of a project)	SiteAddress, SiteCity, SiteState
Client	ClientName, ClientPhone, ClientEmail, ClientCity
Supervisor	SupervisorName, SupervisorPhone
Worker	WorkerName, WorkerPhone, WorkerSkills, WorkerCertifications, WorkerHourlyRate
Supplier	SupplierName, SupplierCity, SupplierPhones
Material (supplied to a project)	MaterialSupplied, MaterialUnitCost
Equipment (used on a project)	EquipmentUsed, EquipmentRentalCost

1.2 Columns repeated across many rows

- All 13 project, site, client and supervisor columns repeat on every row of the same project (P001 appears 3 times, so “Downtown Plaza … Metro Corp … John Carter” is stored 3 times).
- Client details repeat across projects: Metro Corp (P001, P005), City Transit Auth (P002, P007), Retail Ventures (P003, P006).
- Supervisor details repeat: John Carter (P001, P004), Sarah Lee (P002, P005, P007), David Kim (P003, P006).
- Worker phone and hourly rate repeat for every project the worker is on (Mike Ross appears on 4 projects).
- Supplier city and phone list repeat: BuildPro Supplies appears on 4 rows, SteelWorks Inc on 3.

1.3 Multi-valued (non-atomic) cells

Seven columns hold several values separated by `|`:

Column	Example	Note
WorkerSkills	Carpentry|Framing	Independent list
WorkerCertifications	OSHA|First Aid	Independent list
SupplierPhones	617-555-9000|617-555-9001	Independent list
MaterialSupplied + MaterialUnitCost	Concrete|Steel + 120|300	Paired by position: 1st cost belongs to 1st material
EquipmentUsed + EquipmentRentalCost	Crane|Bulldozer + 5000|3000	Paired by position

1.4 Candidate identifiers

- **ProjectID** identifies a project (ProjectName is also unique, so it is an alternate key).
- No ID exists for workers, suppliers, clients or supervisors. Their names are the only identifiers, so later stages introduce surrogate keys (WorkerID, SupplierID, ClientID, SupervisorID).
- In the raw table, **(ProjectID, WorkerName)** is unique, so each raw row is one worker on one project.

1.5 Functional dependencies observed

We confirmed each of these against every row. The build script also re-checks them automatically.

Determinant	Dependent attributes
ProjectID	ProjectName, ProjectType, StartDate, EndDate, SiteAddress, SiteCity, SiteState, ClientName, SupervisorName (and through them client/supervisor details)
ClientName	ClientPhone, ClientEmail, ClientCity
SupervisorName	SupervisorPhone
WorkerName	WorkerPhone, WorkerHourlyRate
SupplierName	SupplierCity
(ProjectID, SupplierName, MaterialSupplied)	MaterialUnitCost (Concrete costs 120, 110, 130, 115 or 140 depending on project and supplier)
(ProjectID, EquipmentUsed)	EquipmentRentalCost (a Crane costs 5000 on P001, 5500 on P002, 6000 on P004, 6500 on P007)

Multi-valued dependencies (one value determines a **set** of values): WorkerName ↠ WorkerSkill, WorkerName ↠ WorkerCertification, SupplierName ↠ SupplierPhone, ProjectID ↠ WorkerName, ProjectID ↠ EquipmentUsed, (ProjectID, SupplierName) ↠ MaterialSupplied.

1.6 Relationships

Relationship	Cardinality
Client – Project	One-to-many (a client commissions many projects, each project has one client)
Supervisor – Project	One-to-many
Project – Worker	Many-to-many (Mike Ross works on P001, P002, P004, P006)
Worker – Skill, Worker – Certification	One-to-many lists (many-to-many with the skill/certification values)
Supplier – Phone	One-to-many
Project – Supplier – Material	Many-to-many-to-many (SteelWorks supplies P001, P004, P007; P004 gets Glass Panels from two suppliers)
Project – Equipment	Many-to-many

1.7 Anomalies caused by the raw design

- **Update anomaly:** Changing Metro Corp's phone means editing 5 rows. Missing one leaves contradictory data. The file already shows this kind of inconsistency: Mike Ross's skills are

listed differently on each row (Carpentry|Framing, Carpentry|Concrete, Concrete|Framing), and Harvey Specter has both “Project Management” and “Management”.

- **Insertion anomaly:** A new supplier or worker cannot be recorded until they are attached to a project.
- **Deletion anomaly:** Deleting project P006 would erase the only record of Southern Concrete (Atlanta, 404-555-6600).

1.8 Assumptions

1. **A worker and a supplier on the same row are not related.** Rows pair a worker with a supplier only because the data was flattened into one table. Nothing says Mike Ross “uses” BuildPro. The real facts are “project has worker” and “project receives material from supplier”.

Evidence: the pairing never means anything consistent. Mike Ross is paired with a different supplier on each of his 4 projects; Louis Litt, an accountant, is paired with a glass supplier on P004 and a steel supplier on P007; and the materials and equipment on a row follow the supplier, not the worker (Mike Ross and Rachel Zane share identical BuildPro data on P001). If the lecturer treated the pairing as meaningful, worker and supplier would have to stay together in one relationship table at 4NF.

2. **Skills and certifications belong to the worker**, not to a single project. The skill lists differ between rows, so we treat a worker’s full skill set as the union of every row (Mike Ross = Carpentry, Concrete, Framing).
3. **Equipment is used by the project.** Rental cost depends on (project, equipment) and is the same whichever supplier row it appears on (Crane on P001 = 5000 on both rows).
4. **Phone numbers are contact details, not identifiers.** Numbers can be shared (e.g. an office switchboard) or reassigned, so we do not use a phone number to identify a supplier. This is discussed again in BCNF.
5. **City does not determine state in general.** In this sample every city has one state, but nationally names repeat (Portland, OR vs Portland, ME), so SiteCity → SiteState is not treated as a dependency.

Stage 1: First Normal Form (1NF)

File: 1NF/construction_1nf.csv (280 rows × 26 columns)

Which columns violated 1NF?

WorkerSkills, WorkerCertifications, SupplierPhones, MaterialSupplied, MaterialUnitCost, EquipmentUsed and EquipmentRentalCost each held more than one value per cell.

How did we make the values atomic?

- Every list was split on |, and the columns were renamed to the singular: WorkerSkill, WorkerCertification, SupplierPhone.
- MaterialSupplied/MaterialUnitCost were split **together** (zipped by position) so that each material stays with its own cost. The same was done for EquipmentUsed/EquipmentRentalCost. Splitting them separately would invent wrong pairs such as “Concrete costs 300”.
- Each raw row was replaced by one row for every combination of its lists. Raw row 1 (2 skills × 2 certifications × 2 phones × 2 materials × 2 equipment) became 32 rows.

Did we create new rows, new tables, or both?

New rows only. We deliberately kept a single table so this stage only shows atomicity. The large redundancy this creates (15 → 280 rows) is the evidence that drives the next stages.

What key identifies each row?

```
PK = (ProjectID, WorkerName, WorkerSkill, WorkerCertification, SupplierName, SupplierPhone,
MaterialSupplied, EquipmentUsed)
```

No smaller set is unique. Removing any one of these columns produces duplicate key values (the script checks this). Each cell now holds a single value, there are no repeating groups and each row is uniquely identified, so the table is in 1NF.

Remaining problems: project, client, worker and supplier data repeats dozens of times (Downtown Plaza appears in 72 rows). All the update, insertion and deletion anomalies are still present.

Stage 2: Second Normal Form (2NF)

Which tables had composite keys?

The single 1NF table has an 8-column composite key (see above).

Which columns depended on only part of the key?

Part of the key (determinant)	Columns that depend only on that part
ProjectID	ProjectName, ProjectType, StartDate, EndDate, SiteAddress, SiteCity, SiteState, ClientName, ClientPhone, ClientEmail, ClientCity, SupervisorName, SupervisorPhone
WorkerName	WorkerPhone, WorkerHourlyRate
SupplierName	SupplierCity
(ProjectID, SupplierName, MaterialSupplied)	MaterialUnitCost
(ProjectID, EquipmentUsed)	EquipmentRentalCost

No non-key column depends on the whole 8-column key, so every non-key column was moved out.

Which data did we move into separate tables?

File	Primary key	Contents
projects.csv	ProjectID	Project, site, client and supervisor columns (7 rows)
workers.csv	WorkerID	WorkerName, WorkerPhone, WorkerHourlyRate (5 rows)
suppliers.csv	SupplierID	SupplierName, SupplierCity (6 rows)
project_material_supply.csv	(ProjectID, SupplierID, MaterialSupplied)	MaterialUnitCost (16 rows)
project_equipment.csv	(ProjectID, EquipmentUsed)	EquipmentRentalCost (13 rows)
project_resource_links.csv	all 8 columns	The original key columns only (280 rows). This keeps the decomposition lossless.

What primary and foreign keys did we introduce?

- **New surrogate keys:** WorkerID (W01–W05) replaces WorkerName and SupplierID (S01–S06) replaces SupplierName. Names are not reliable identifiers because two people or companies can share a name.
- **Foreign keys:** project_material_supply.ProjectID → projects, project_material_supply.SupplierID → suppliers, project_equipment.ProjectID → projects, and in project_resource_links: ProjectID → projects, WorkerID → workers, SupplierID → suppliers.

Why is the design now in 2NF?

Every table is in 1NF. Tables with a single-column key (projects, workers, suppliers) cannot have partial dependencies. In project_material_supply, MaterialUnitCost needs all three key columns, because the same material has different prices for different projects and suppliers. In project_equipment, rental cost needs both project and equipment. project_resource_links has no non-key columns at all. So no non-key attribute depends on only part of a key.

Remaining problem: projects.csv still repeats client and supervisor details (Metro Corp's phone and email appear twice; Sarah Lee's phone three times).

Stage 3: Third Normal Form (3NF)

Which transitive dependencies did we find?

In projects.csv (key ProjectID):

```
ProjectID → ClientName → ClientPhone, ClientEmail, ClientCity
ProjectID → SupervisorName → SupervisorPhone
```

Which non-key attributes depended on other non-key attributes?

- ClientPhone, ClientEmail and ClientCity depend on ClientName (non-key).
- SupervisorPhone depends on SupervisorName (non-key).

We checked the other tables too. In workers, suppliers, project_material_supply and project_equipment, each non-key column depends directly on the key. SiteCity → SiteState was considered and rejected (Assumption 5). SiteAddress, SiteCity and SiteState together describe where one project is built and depend directly on ProjectID.

Which new tables did we create?

File	Primary key	Columns
clients.csv	ClientID (C01–C04)	ClientName, ClientPhone, ClientEmail, ClientCity
supervisors.csv	SupervisorID (SV01–SV03)	SupervisorName, SupervisorPhone
projects.csv (modified)	ProjectID	ProjectName, ProjectType, StartDate, EndDate, SiteAddress, SiteCity, SiteState, ClientID, SupervisorID

How did we use foreign keys to preserve relationships?

projects.ClientID → clients.ClientID and projects.SupervisorID → supervisors.SupervisorID. Joining projects to clients and supervisors on these keys rebuilds the exact 2NF projects table, so no information was lost. Metro Corp's phone is now stored once instead of on every project.

The other five 2NF tables carry over unchanged. 3NF folder: projects, clients, supervisors, workers, suppliers, project_material_supply, project_equipment, project_resource_links.

Why is the design now in 3NF?

Every table is in 2NF, and in every table each non-key attribute depends on the key, the whole key and nothing but the key. No non-key attribute determines another non-key attribute.

Stage 4: Boyce-Codd Normal Form (BCNF)

BCNF requires that for **every** non-trivial functional dependency X → Y, X is a candidate key (or superkey). Unlike 3NF, there is no exception when Y is part of a key.

Which functional dependencies did we check?

Method. For each 3NF table we (1) listed every non-trivial FD that holds, including ones that start from an alternate key such as a name, (2) computed the closure X^+ of each determinant X, meaning every column X determines, and (3) marked X as a superkey only if X^+ contained every column of the table. Example for projects: $\{ProjectID\}^+$ = {ProjectID, ProjectName, ProjectType, StartDate, EndDate, SiteAddress, SiteCity, SiteState, ClientID, SupervisorID}, which is every column, so ProjectID is a key. $\{ClientID\}^+$ inside projects is just {ClientID}, because ClientID → ProjectID does not hold (C01 has P001 and P005), so ClientID determines nothing in that table and cannot cause a violation.

Results for every table:

Table	Functional dependencies	Candidate keys	BCNF
projects	ProjectID → all; ProjectName → all	ProjectID, ProjectName	Yes
clients	ClientID → all; ClientName → all; ClientEmail → all	ClientID, ClientName, ClientEmail (ClientPhone too, if the business treats it as unique)	Yes
supervisors	SupervisorID → all; SupervisorName → all	SupervisorID, SupervisorName	Yes
workers	WorkerID → all; WorkerName → all	WorkerID, WorkerName	Yes
suppliers	SupplierID → all; SupplierName → all	SupplierID, SupplierName	Yes
project_material_supply	(ProjectID, SupplierID, MaterialSupplied) → MaterialUnitCost	(ProjectID, SupplierID, MaterialSupplied)	Yes
project_equipment	(ProjectID, EquipmentUsed) → EquipmentRentalCost	(ProjectID, EquipmentUsed)	Yes
project_resource_links	Only trivial FDs (all-key table)	All 8 columns	Yes

Did we find any determinant that was not a candidate key?

No. Because 3NF and BCNF only differ when a non-key column determines part of a key, we tested every candidate dependency of that kind and looked for two rows that disprove it. Each one below fails on real data (our build script finds and prints these counterexamples automatically):

Rule we tested	Where it would break BCNF	Counterexample in the data	Holds
SupplierID → MaterialSupplied	project_material_supply	BuildPro (S01) supplies Concrete and Steel	No
(ProjectID, MaterialSupplied) → SupplierID	project_material_supply	P004 buys Glass Panels from SteelWorks (S02) and Global Glass (S04)	No
SupervisorID → ProjectType	projects	Sarah Lee (SV02) runs Infrastructure and Commercial projects	No
ClientID → SupervisorID	projects	Metro Corp (C01) has John Carter and Sarah Lee	No

Worked example: what a BCNF violation would look like here

Suppose the business had these two rules: “each supplier sells only one material” (SupplierID → MaterialSupplied) and “each project buys each material from only one supplier” ((ProjectID, MaterialSupplied) → SupplierID). Then project_material_supply would have two overlapping candidate keys, (ProjectID, MaterialSupplied) and (ProjectID, SupplierID). Every column would be part of some key, so the table would pass 3NF. It would still fail BCNF, because SupplierID → MaterialSupplied has a determinant that is not a key. The fix would be to decompose it into:

supplier_material(**SupplierID**, MaterialSupplied)
project_supplier(**ProjectID**, **SupplierID**, MaterialUnitCost)

This join is lossless because SupplierID, the shared column, is the key of supplier_material. However, both rules are disproved by the first two rows of the table above, so this decomposition would be **wrong** for our data: it would claim BuildPro only ever sells one material. That is why we keep project_material_supply as it is.

Other dependencies we considered:

- **(ProjectID, MaterialSupplied) → MaterialUnitCost?** It happens to hold in this sample, but only one project buys the same material from two suppliers, so it is a coincidence rather than a rule. In practice prices are set by the supplier, so we keep SupplierID in the determinant.
- **SupplierPhone → SupplierID** in project_resource_links? Each phone appears with one supplier in the sample, but under Assumption 4 a phone is not an identifier, so this is not a dependency we enforce. If the business decided phone numbers are unique, the final supplier_phones table would simply gain SupplierPhone as an alternate key and would still be in BCNF.
- **SiteCity → SiteState?** Rejected (Assumption 5).

Which tables violated BCNF, and how did we decompose them?

None. In every 3NF table, every determinant is a candidate key, so no decomposition was needed. The BCNF folder therefore contains the same 8 files as 3NF. This is expected: a table can be in 3NF but not BCNF only when it has overlapping composite candidate keys, and the worked example above shows the data rules out the only place where that could have happened.

What candidate keys exist after this stage?

As listed in the table above. Alternate keys (ProjectName, ClientName, ClientEmail, SupervisorName, WorkerName, SupplierName) should get UNIQUE constraints in a real database.

Why is the design in BCNF?

For every non-trivial functional dependency in every table, the left-hand side is a candidate key.

Remaining problem: project_resource_links is in BCNF but still has 280 rows of redundancy. That redundancy comes from multi-valued dependencies, not functional ones, and 4NF deals with it.

Stage 5: Fourth Normal Form (4NF)

A table is in 4NF when it is in BCNF and, for every non-trivial multi-valued dependency X ↠ Y, X is a superkey.

Which multi-valued dependencies did we identify?

In project_resource_links(ProjectID, WorkerID, WorkerSkill, WorkerCertification, SupplierID, SupplierPhone, MaterialSupplied, EquipmentUsed):

MVD	Meaning
WorkerID ↠ WorkerSkill	A worker has a set of skills, independent of anything else
WorkerID ↠ WorkerCertification	A worker has a set of certifications, independent of their skills
SupplierID ↠ SupplierPhone	A supplier has a set of phone numbers
ProjectID ↠ WorkerID	A project has a set of workers
ProjectID ↠ EquipmentUsed	A project uses a set of equipment
(ProjectID, SupplierID) ↠ MaterialSupplied	A supplier delivers a set of materials to a project

None of these determinants is a superkey of the 8-column table, so it violates 4NF.

Which independent lists were mixed together?

Each row combines one skill, one certification, one supplier phone, one material and one equipment item, even though these facts have nothing to do with each other. Example for Mike Ross on P001: 2 skills × 2 certifications × 2 BuildPro phones × 2 materials × 2 equipment = 32 rows to say what 10 simple facts would say. Adding a third certification for Mike would need 16 new rows for P001 alone, which is the classic 4NF update anomaly.

How did we separate them?

We split the table along each MVD, keeping the determinant together with each independent list:

New file	Primary key	Rows	Foreign keys
project_workers.csv	(ProjectID, WorkerID)	15	ProjectID → projects, WorkerID → workers
worker_skills.csv	(WorkerID, Skill)	15	WorkerID → workers
worker_certifications.csv	(WorkerID, Certification)	12	WorkerID → workers
supplier_phones.csv	(SupplierID, SupplierPhone)	8	SupplierID → suppliers

The remaining projections, (ProjectID, SupplierID, MaterialSupplied) and (ProjectID, EquipmentUsed), are **already stored** as the keys of project_material_supply and project_equipment, created in 2NF. We confirmed that their values match exactly, so no new table is needed. project_resource_links was then dropped.

What are the primary keys of the new relationship tables?

Each one is the full composite of its two columns (see table above). Both columns are needed: a worker has many skills, and a skill is held by many workers.

Why is the final design in 4NF?

- Every table is in BCNF (the 7 carried-over tables were checked in the BCNF stage; the 4 new ones are all-key and have no non-trivial FDs).
- Each two-column relationship table holds exactly one multi-valued fact. Its only MVD is trivial, because the determinant and the dependent set together make up the whole table.
- In the entity tables and in project_material_supply and project_equipment, every MVD comes from an FD whose determinant is a candidate key.
- No table stores two independent lists side by side any more.

Final file list (4NF/): projects, clients, supervisors, workers, suppliers, project_workers, worker_skills, worker_certifications, supplier_phones, project_material_supply, project_equipment (11 files).

Result: 280 redundant rows became 11 small tables with 104 rows in total. Each fact is stored exactly once, and every insertion, update and deletion anomaly from Step 1 is gone. For example, Southern Concrete stays in suppliers even if P006 is deleted.

Primary and Foreign Key Summary (Final Design)

Table	Type	Primary key	Foreign keys
projects	Entity	ProjectID	ClientID → clients; SupervisorID → supervisors
clients	Entity	ClientID	–
supervisors	Entity	SupervisorID	–
workers	Entity	WorkerID	–
suppliers	Entity	SupplierID	–
project_workers	Relationship (M:N)	(ProjectID, WorkerID)	ProjectID → projects; WorkerID → workers
worker_skills	Multi-valued attribute	(WorkerID, Skill)	WorkerID → workers
worker_certifications	Multi-valued attribute	(WorkerID, Certification)	WorkerID → workers
supplier_phones	Multi-valued attribute	(SupplierID, SupplierPhone)	SupplierID → suppliers
project_material_supply	Relationship (M:N:N)	(ProjectID, SupplierID, MaterialSupplied)	ProjectID → projects; SupplierID → suppliers
project_equipment	Relationship (M:N)	(ProjectID, EquipmentUsed)	ProjectID → projects

Relationship diagram (text form)

```
clients 1 —< projects >— 1 supervisors
projects 1 —< project_workers >— 1 workers 1 —< worker_skills
                                  workers 1 —< worker_certifications
projects 1 —< project_material_supply >— 1 suppliers 1 —< supplier_phones
projects 1 —< project_equipment
```

Data-quality notes for the client

- Harvey Specter's skills include both "Project Management" and "Management". These are probably the same skill and should be confirmed and merged in worker_skills.
- Materials "Steel" (BuildPro, P001) and "Steel Beams" (SteelWorks) are kept as different materials, as recorded.
- Possible extension: Material and Equipment lookup tables (MaterialID, EquipmentID) would prevent spelling variants. They are not required by any normal form here because those tables would have no other attributes.

Group Roles and Contributions

Member	Role	Responsibilities
Tito	Design lead and author	Analysed the raw data (Step 1); designed every normalization stage and chose the keys; wrote the script that generates and checks all CSV files; wrote this explanation document; integrated the group's reviews and assembled the final submission.
Cedric	1NF & 2NF verification	Traced sample raw rows through the 1NF and 2NF files to confirm no data was lost or invented; confirmed row counts and composite keys; presents 1NF and 2NF.
Ange	3NF, BCNF & assumptions review	Reviewed the transitive dependencies, the BCNF checks and the counterexamples against the CSV files; reviewed and challenged the assumptions in section 1.8; presents 3NF and BCNF.
Christian	4NF & diagram	Checked the 4NF tables against the raw data; drew the entity-relationship diagram of the final design with primary and foreign keys; presents 4NF and the final design.